

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Let's write a function that...

Takes the name of a text file, and replaces its contents with the sarcastic version of the text.

We'll need to:

- Read all the content of a given text file
- Sarcastify the string (we did that yesterday -- capitalize a random half of the letters)
- Overwrite the text file with a new file that has the same name, with the sarcastic contents

Read and write data from text files

Read data from a file:

```
with open('story.txt', 'r', encoding="utf-8") as file:  
    for line in file.readlines():  
        print(line)
```

Write to a file instead of reading --> use an option other than `'r'`:

- `open('story.txt', 'r')` : read the file
- `open('story.txt', 'w')` : write the file (overwrite it if it already exists)
- `open('story.txt', 'a')` : append to the end of the file (and create the file if it doesn't exist)

Write to the file using `file.write("Line to write to file")`.

None

`None` works like a value that represents the absence of a value.

```
bodyguard_name: str = None # doesn't have a value -- I don't have a bodyguard
```

It's different from "" or 0 (see [Null Island](#))

Can store `None` in a list

```
grades: List[int] = [5, None, 0]
```

Cannot add `None` to a number or string

- `None + "hi"` does not work
- `len(None)` does not work

Use `Optional` to specify that `None` is possible

```
from typing import Optional

def get_number_or_None(hopefully_a_number: str) -> Optional[int]:
    try:
        return int(hopefully_a_number)
    except ValueError:
        return None
```

In the function above, the result is either the `int` value converted from the string input, or `None` because the string input does not represent an integer.

Poll: What's wrong here?

```
def get_oldest(ages: list[int]) -> int:  
    if not ages:  
        return None  
    return max(ages)
```

1. The return type needs to be `Optional[int]`
2. The `ages` argument type needs to be `Optional[list[int]]`
3. The `ages` argument type needs to be `list[Optional[int]]`
4. The function doesn't handle the case where the `ages` list is empty

Poll: What is output?

```
def find_even(numbers: list[int]) -> Optional[int]:  
    for n in numbers:  
        if n % 2 == 0:  
            return n  
  
result = find_even([1, 3, 5])  
print(result)  
print(type(result))
```

1. (Nothing) \ `<class 'NoneType'>`
2. (Nothing) \ `None`
3. `None` \ `None`
4. `None` \ `<class 'NoneType'>`

Poll: What is output?

```
def double(x: int) -> int:  
    y = x * 2  
  
print(double(5) + 1)
```

1. 11

2. None

3. double(5) + 1

4. TypeError: NoneType has no attribute...

Using `try` / `except` safely

A control structure that we have not introduced until now:

```
a: int = 4
b: int = 0

try:
    result = a / b
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

- Allows us to try to run risky code
- If an error is raised during that risky code, it jumps to the corresponding `except` block
- Only use it when absolutely necessary -- do not avoid fixing bugs with `try`

Places where `try / except` is commonly used:

- Converting values

```
def get_user_age() -> int:
    """Get a numerical age from the user"""
    user_input: str = input("Enter your age: ")
    try:
        age: int = int(user_input)
        return age
    except ValueError:
        print("Please enter a valid number")
        return -1

def parse_json_safely(json_string: str) -> Any:
    """Convert data from JSON to a readable format"""
    try:
        return json.loads(json_string)
    except json.JSONDecodeError as e:
        print(f"Invalid JSON: {e}")
        return {}
```

- Operations that rely on external things like network requests or database operations
- File I/O (though `with` is better)
- `try / except` is an acceptable alternative to the built-in `pytest.raises()`

Keywords in a `try` / `except` block:

- Each error type gets its own `except`
 - Error types as specific as possible (`ValueError` , not `Error`)
 - Okay to have multiple `except` s for the same `try`
 - One `except` block can handle multiple errors, if they require the same process:
`except (ValueError, TypeError) as e:`
- Inside an `except` block, we may choose to `raise` a different error
- If there is a `finally` at the end, it is always run (whether the `try` was fully executed, or it jumped to the `except`)
- If there is an `else` at the end, then it is run only if the `try` was fully successful

Poll: What is output?

```
def noodle(hopefully_a_number: str) -> None:
    try:
        num: int = int(hopefully_a_number)
        print('Cats rule')
    except AssertionError as e:
        print(f'{hopefully_a_number} is not a number')

noodle('hello')
```

1. Cats rule
2. hello is not a number
3. Cats rule
hello is not a number
4. No output - it raises the error

Poll: What is output?

```
def noodle(hopefully_a_number: str) -> None:
    try:
        num: int = int(hopefully_a_number)
        print('Cats rule')
    except ValueError as e:
        print(f'{hopefully_a_number} is not a number')

noodle('hello')
```

1. Cats rule
2. hello is not a number
3. Cats rule
hello is not a number
4. No output - it raises the error

Poll:

- 1. What is your main takeaway from today?**
- 2. What would you like to revisit next time?**